

SmartLoan

Sistem Peminjaman Aset Multi-Cabang

Studi kasus latihan Keycloak untuk backend NestJS + TypeORM

Stack	NestJS, TypeORM, PostgreSQL, Keycloak (docker-compose)
Fokus latihan	Realm/client config, validasi JWT, RBAC, branch scoping, Admin REST API, refresh dan logout
Format kerja	8 fase, satu fase satu commit/PR, tiap fase punya kriteria PASS/FAIL

Cara membaca dokumen ini. Bagian 1–5 menjelaskan *apa yang dibangun*: gambaran besar, entity, alur, dan endpoint. Bagian 6–12 adalah *spesifikasi kerja*: konfigurasi Keycloak, requirement backend, batasan, fase, dan pertanyaan validasi. Kalau bingung, baca berurutan dari bagian 1 dan jangan lompat ke bagian 6.

1. Brief dalam Bahasa Sederhana

SmartLoan adalah aplikasi internal untuk meminjam aset kantor (laptop, proyektor, mobil operasional). Ada 3 cabang: Jakarta (JKT), Surabaya (SBY), dan Malang (MLG). Karyawan mengajukan pinjaman, staf cabang menyetujui dan mengurus serah-terima, manajer cabang mengelola daftar aset, dan super admin mengelola user.

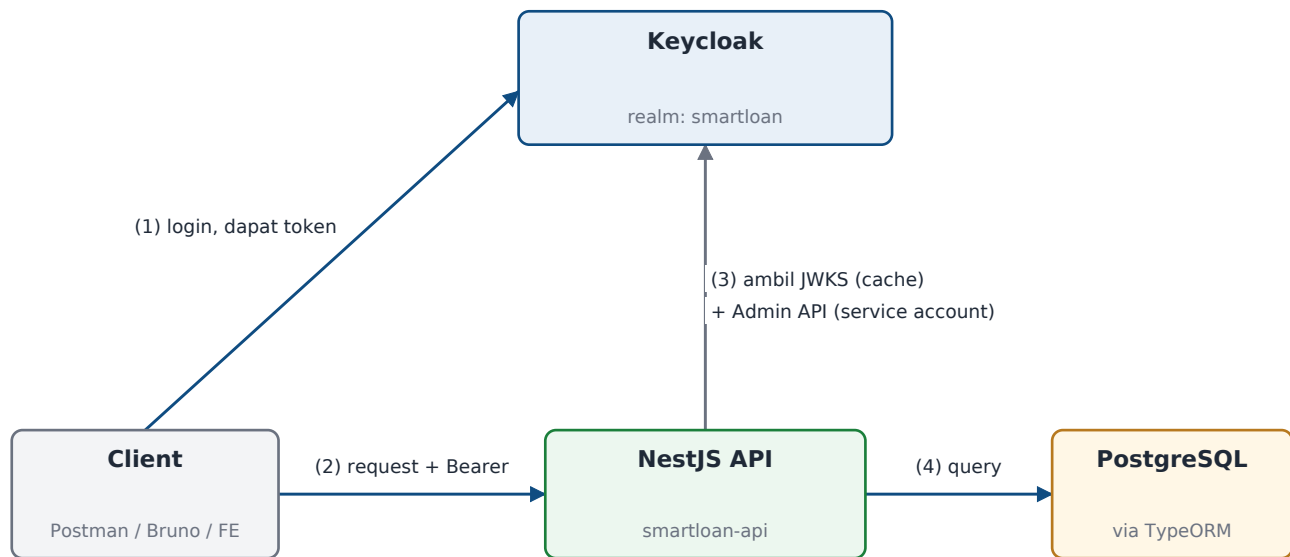
Analogi gedung kantor. Keycloak adalah resepsionis: memeriksa siapa kamu (login), lalu memberi kartu akses (token) yang memuat namamu, jabatanmu (role), dan cabangmu. **Backend NestJS** adalah pintu-pintu ruangan: tidak peduli passwordmu, hanya memeriksa apakah kartunya asli dan apakah jabatan serta cabangmu boleh masuk. **PostgreSQL** adalah isi ruangan: data aset dan pinjaman.

Tugasmu: membangun pintu-pintu itu (NestJS + TypeORM) dan mengatur si resepsionis (Keycloak).

Siapa memegang apa

Komponen	Bertanggung jawab atas	Tidak melakukan
Keycloak	Password dan login, penerbitan token, role, group/cabang, sesi (refresh, logout), proteksi brute-force.	Tidak tahu apa itu aset atau loan.
NestJS API	Memverifikasi token, menegakkan role dan scope cabang, aturan bisnis peminjaman.	Tidak menyimpan password. Tidak punya endpoint login/refresh/logout.
PostgreSQL	Data domain: cabang, profil user (salinan minimal), aset, loan.	Tidak menyimpan password maupun role.

Gambaran arsitektur



(1) Client login langsung ke Keycloak dan menerima access token. **(2)** Client memanggil API dengan token di header `Authorization: Bearer ....` **(3)** API memverifikasi tanda tangan token memakai public key (JWKS) dari Keycloak. Key ini diambil lalu di-cache, bukan ditanyakan tiap request. API juga memakai service account untuk memanggil Admin API. **(4)** API membaca dan menulis data domain di PostgreSQL.

Konsep Keycloak yang dilatih dan di mana muncul

Konsep Keycloak	Di studi kasus ini
Realm	smartloan: satu "dunia" berisi semua user, role, dan client.
Client public	smartloan-web: aplikasi yang meminta login (Authorization Code + PKCE).
Client confidential	smartloan-api: identitas backend, sekaligus tujuan (audience) token.
Realm role dan composite role	BORROWER, STAFF, BRANCH_MANAGER (mencakup STAFF), SUPER_ADMIN.
Group	/branches/JKT, /branches/SBY, /branches/MLG: penanda cabang user.
Protocol mapper	Menambahkan claim <code>branch_code</code> dan <code>aud</code> ke token.
JWT dan JWKS	AuthGuard memverifikasi signature, iss, aud, exp.
Service account dan Admin REST API	POST /admin/users dan PATCH /admin/users/:id/status.
Refresh token, sesi, logout	Fase 7. Dipanggil langsung ke Keycloak, bukan lewat backend.
Brute-force, password policy, required action	Pengaturan realm dan onboarding user baru (wajib ganti password).

2. Aktor, Role, dan User Uji

Role (realm role)	Siapa	Hak akses
BORROWER	Semua karyawan	Mengajukan pinjaman, melihat dan membatalkan loan miliknya sendiri.
STAFF	Staf gudang/GA cabang	Approve, reject, serah-terima, dan proses pengembalian hanya di cabangnya .
BRANCH_MANAGER	Manajer cabang	Semua hak STAFF + CRUD aset cabangnya. Composite role yang mencakup STAFF.
SUPER_ADMIN	Admin pusat	Lintas semua cabang + kelola user lewat Admin API.

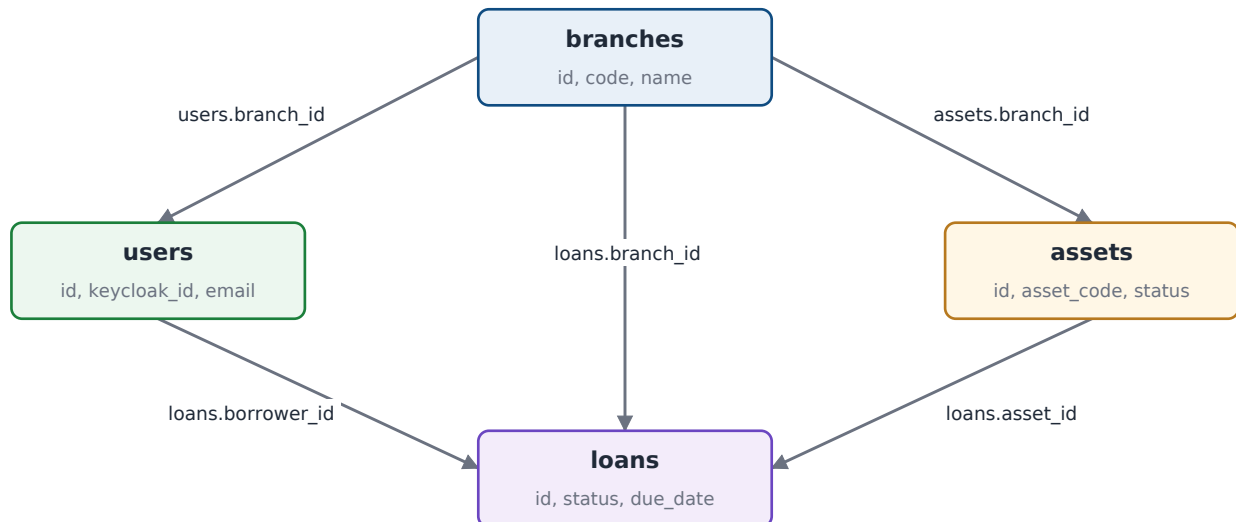
Aturan multi-role. Satu user boleh punya beberapa role. Hak aksesnya adalah **gabungan** semua role, dan scope terluas yang menang. Contoh: Rina adalah STAFF sekaligus BORROWER. Ia boleh meminjam aset dan boleh memproses loan orang lain, tetapi tidak boleh menyetujui loan miliknya sendiri (aturan R6).

User uji (seed di realm import)

Nama	Username	Role	Group	Status
Sari Wulandari	sari	SUPER_ADMIN	tanpa cabang	aktif
Budi Santoso	budi	BORROWER, BRANCH_MANAGER	/branches/JKT	aktif
Rina Putri	rina	BORROWER, STAFF	/branches/JKT	aktif
Tono Hartono	tono	BORROWER, STAFF	/branches/SBY	aktif
Andi Pratama	andi	BORROWER	/branches/JKT	aktif
Dewi Lestari	dewi	BORROWER	/branches/SBY	aktif
Eko Nugroho	eko	BORROWER	/branches/JKT	disabled

3. Entity dan Field

Ada 4 entity. Semua memakai primary key UUID, nama tabel bahasa Inggris, nilai enum SCREAMING_SNAKE_CASE, dan index bergaya `@Index('IDX_TABLE_COLUMN', [...])`. Gunakan **migration**, jangan `synchronize: true`.



Garis menunjukkan foreign key (satu ke banyak). Selain `loans.borrower_id`, tabel `loans` punya tiga FK ke `users` untuk mencatat siapa yang memproses (lihat tabel `loans`).

3.1 branches

Field	Tipe	Null?	Keterangan
id	uuid	tidak	Primary key.
code	varchar(10)	tidak	Unique. JKT, SBY, MLG. Harus sama dengan nama group di Keycloak dan isi claim <code>branch_code</code> .
name	varchar(100)	tidak	Nama cabang.
address	text	ya	Alamat cabang.
created_at	timestampz	tidak	Diisi otomatis. <code>updated_at</code> juga.

3.2 users (profil lokal)

Field	Tipe	Null?	Keterangan
id	uuid	tidak	Primary key internal aplikasi. Dipakai sebagai FK di tabel lain.
keycloak_id	uuid	tidak	Unique. Diisi dari claim sub di token. Ini kunci penghubung ke Keycloak.
email	varchar(255)	tidak	Salinan dari token untuk tampilan. Bukan sumber kebenaran.
full_name	varchar(150)	tidak	Salinan dari claim name.
branch_id	uuid FK	ya	Ke branches. Kosong untuk SUPER_ADMIN. Hanya cache untuk join/laporan; scoping tetap memakai claim token.
is_active	boolean	tidak	Default true. Dipakai saat user di-disable lewat Admin API.
last_login_at	timestampz	ya	Diperbarui saat request pertama pada sesi baru (opsional).
created_at	timestampz	tidak	Juga updated_at.

Tidak ada kolom password maupun role di tabel ini. Keduanya milik Keycloak.

3.3 assets

Field	Tipe	Null?	Keterangan
id	uuid	tidak	Primary key.
branch_id	uuid FK	tidak	Ke branches. Cabang pemilik aset.
asset_code	varchar(30)	tidak	Unique. Contoh JKT-LPT-001.
name	varchar(150)	tidak	Contoh: Laptop Dell Latitude 5440.
category	enum	tidak	LAPTOP, PROJECTOR, VEHICLE, OTHER.
status	enum	tidak	AVAILABLE, RESERVED, BORROWED, MAINTENANCE, RETIRED. Default AVAILABLE.
description	text	ya	Catatan tambahan.
created_at	timestampz	tidak	Juga updated_at.
deleted_at	timestampz	ya	Soft delete (@DeleteDateColumn).

Index: IDX_ASSETS_BRANCH_ID_STATUS pada (branch_id, status).

3.4 loans

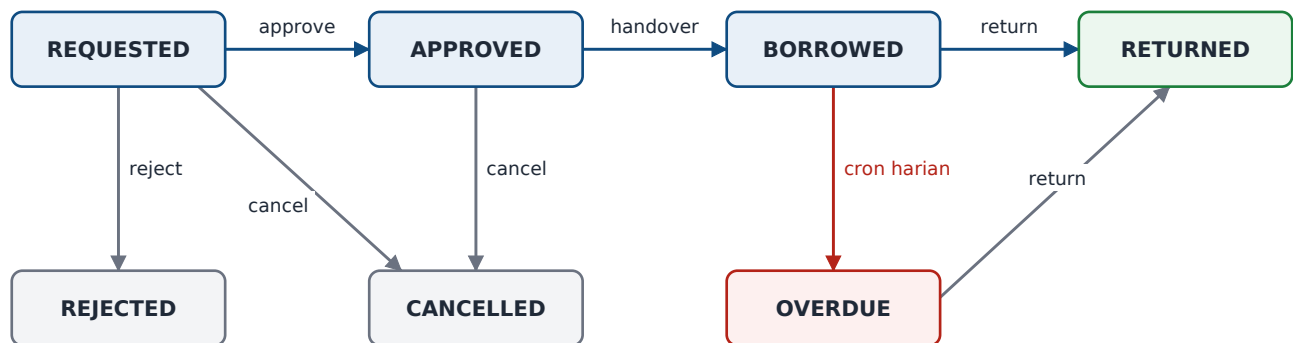
Field	Tipe	Null?	Keterangan
id	uuid	tidak	Primary key.
loan_number	varchar(20)	tidak	Unique. Dibuat aplikasi, contoh LN-202609-0001.
asset_id	uuid FK	tidak	Ke assets.
branch_id	uuid FK	tidak	Salinan assets.branch_id supaya query scoping cepat tanpa join.
borrower_id	uuid FK	tidak	Ke users. Diisi dari token, tidak pernah dari body request.
status	enum	tidak	REQUESTED, APPROVED, REJECTED, BORROWED, RETURNED, OVERDUE, CANCELLED. Default REQUESTED.
purpose	text	tidak	Tujuan peminjaman.
due_date	date	tidak	Batas pengembalian.
approved_by_id	uuid FK	ya	Ke users. Yang menyetujui. Plus approved_at (timestampz, ya).
handed_over_by_id	uuid FK	ya	Ke users. Yang menyerahkan aset. Plus borrowed_at (timestampz, ya).
received_by_id	uuid FK	ya	Ke users. Yang menerima pengembalian. Plus returned_at (timestampz, ya).
return_notes	text	ya	Kondisi aset saat dikembalikan.
reject_reason	text	ya	Alasan penolakan.
cancel_reason	text	ya	Alasan pembatalan.
created_at	timestampz	tidak	Juga updated_at.

Index yang disarankan

- IDX_LOANS_BRANCH_ID_STATUS (branch_id, status): daftar loan per cabang.
- IDX_LOANS_BORROWER_ID (borrower_id): daftar loan milik user.
- IDX_LOANS_DUE_DATE (due_date): dipakai job overdue.
- IDX_LOANS_ASSET_ID_ACTIVE (asset_id): **partial unique index** dengan where: "status IN ('APPROVED', 'BORROWED', 'OVERDUE')". Database sendiri menolak dua loan aktif untuk satu aset.

4. Alur dari Awal sampai Akhir

Siklus hidup status loan



Biru = status aktif, abu-abu = akhir tanpa pemulihan, merah = terlambat. OVERDUE dibuat oleh job harian (BORROWED dengan due_date sebelum hari ini), bukan oleh endpoint. Transisi di luar panah di atas harus ditolak dengan 409.

Status aset mengikuti status loan

Kejadian pada loan	Status loan	Status aset
Pengajuan dibuat	REQUESTED	AVAILABLE (belum berubah)
Disetujui	APPROVED	AVAILABLE → RESERVED
Diserahkan ke peminjam	BORROWED	RESERVED → BORROWED
Dikembalikan (kondisi baik)	RETURNED	BORROWED → AVAILABLE
Dikembalikan (perlu perbaikan)	RETURNED	BORROWED → MAINTENANCE
Dibatalkan setelah disetujui	CANCELLED	RESERVED → AVAILABLE
Ditolak / dibatalkan sebelum disetujui	REJECTED / CANCELLED	tidak berubah

Alur A: Persiapan (sekali di awal)

#	Aktor	Aksi	Yang terjadi di sistem
1	Kamu	Jalankan docker compose up	Keycloak dan Postgres hidup. Realm smartloan ter-import otomatis: client, role, group, dan 7 user uji.
2	Kamu	Jalankan migration + seed	4 tabel terbentuk. Tabel branches terisi JKT, SBY, MLG, dan tiap cabang punya beberapa assets contoh. Tabel users masih kosong .
3	Kamu	Start NestJS	Backend siap. Public key (JWKS) Keycloak diambil pada request pertama lalu di-cache.

Alur B: Peminjaman normal (Andi meminjam laptop JKT-LPT-001)

#	Aktor	Aksi	Yang terjadi di sistem
1	Andi	Login lewat Keycloak (Authorization Code + PKCE)	Keycloak memeriksa password, lalu menerbitkan access token (5 menit) dan refresh token. Backend belum terlibat. Token memuat sub, role BORROWER, branch_code=JKT, aud=smartloan-api.
2	Andi	GET /me dengan Bearer token	AuthGuard memverifikasi signature, iss, aud, exp. JIT provisioning: belum ada baris users, maka dibuat (keycloak_id = sub). Respons berisi profil + role + cabang.
3	Andi	GET /assets?status=AVAILABLE	Hanya aset cabang JKT yang tampil, karena scope diambil dari claim branch_code.
4	Andi	POST /loans {asset_id, due_date, purpose}	Backend cek aturan R1-R5. Loan dibuat berstatus REQUESTED , borrower_id dari token. Aset masih AVAILABLE.
5	Rina	GET /loans?status=REQUESTED	Rina (STAFF JKT) melihat pengajuan Andi. Tono (STAFF SBY) tidak melihatnya, dan GET /loans/:id oleh Tono menghasilkan 404.
6	Rina	PATCH /loans/:id/approve	Dalam satu transaksi: kunci baris aset, cek masih AVAILABLE, loan → APPROVED , aset → RESERVED , approved_by_id = Rina.
7	Rina	PATCH /loans/:id/handover saat Andi datang mengambil	Loan → BORROWED , aset → BORROWED , borrowed_at dan handed_over_by_id terisi.
8	Rina	PATCH /loans/:id/return {return_notes, needs_maintenance}	Loan → RETURNED . Aset → AVAILABLE (atau MAINTENANCE bila perlu perbaikan). returned_at dan received_by_id terisi.

Alur C: Percabangan dan kasus tepi

#	Aktor	Aksi	Yang terjadi di sistem
1	Rina	PATCH /loans/:id/reject {reason}	REQUESTED → REJECTED . Aset tidak berubah.
2	Andi	PATCH /loans/:id/cancel	REQUESTED atau APPROVED → CANCELLED . Bila aset sempat RESERVED, kembali AVAILABLE.
3	Job harian	Berjalan otomatis (tanpa endpoint)	Loan BORROWED dengan due_date sebelum hari ini → OVERDUE . Andi tidak bisa mengajukan loan baru (R5), tetapi tetap bisa mengembalikan.
4	Dewi	POST /loans untuk aset JKT	Dewi berasal dari SBY, maka aset JKT dianggap tidak ada → 404 (R1).
5	Rina	Mengajukan loan lalu menyetujuinya sendiri	403 (R6). Ia punya role STAFF, tetapi tidak boleh memproses loan miliknya.
6	2 staf	Approve dua loan berbeda untuk aset yang sama, bersamaan	Satu berhasil, satu 409 . Dijaga oleh pessimistic lock + partial unique index.

Alur D: Mengelola user (Sari sebagai SUPER_ADMIN)

#	Aktor	Aksi	Yang terjadi di sistem
1	Sari	Login	Token berisi role SUPER_ADMIN.
2	Sari	POST /admin/users {email, full_name, branch_code, roles, temporary_password}	Backend mengambil token service account (dari cache). Lewat Admin API: buat user, set password sementara + required action UPDATE_PASSWORD, assign role, masukkan ke group cabang. Lalu insert baris users lokal. Bila insert lokal gagal, batalkan pembuatan di Keycloak (kompensasi).
3	Karyawan baru	Login pertama	Keycloak memaksa ganti password. Setelah itu GET /me menemukan profil yang sudah ada (tidak dibuat ulang).
4	Sari	PATCH /admin/users/:id/status {enabled: false}	Keycloak: user di-disable dan sesi aktif dicabut. Lokal: is_active = false. Refresh token tidak lagi berfungsi. Access token lama masih valid sampai habis (5 menit) kecuali guard ikut memeriksa is_active (bahan pertanyaan validasi 1).

Alur E: Sesi (dilakukan langsung ke Keycloak)

#	Aktor	Aksi	Yang terjadi di sistem
1	Client	Access token habis, API menjawab 401	Client menukar refresh token ke Keycloak dan mendapat token baru. Backend tidak punya endpoint refresh.
2	Andi	Logout	Client memanggil logout endpoint Keycloak dengan refresh token. Sesi dicabut, refresh token tidak bisa dipakai lagi.

5. Endpoint: Fungsi dan Aturan**Aturan bisnis**

Kode	Aturan
R1	Peminjam hanya boleh meminjam aset di cabangnya sendiri (claim branch_code = asset.branch).
R2	Aset harus berstatus AVAILABLE saat pengajuan dan saat approve.
R3	due_date minimal besok dan maksimal 14 hari dari hari ini.
R4	Satu peminjam maksimal 2 loan aktif (REQUESTED, APPROVED, BORROWED, OVERDUE).
R5	Peminjam yang punya loan OVERDUE tidak boleh mengajukan loan baru.
R6	Yang menyetujui/menolak tidak boleh peminjam dari loan itu sendiri.
R7	STAFF dan BRANCH_MANAGER hanya memproses loan cabangnya. SUPER_ADMIN semua cabang.
R8	Hanya transisi status pada diagram yang diizinkan. Selainnya ditolak.
R9	Approve bersifat atomik: transaksi + pessimistic write lock pada baris aset.

Konvensi kode respons

Kode	Kapan dipakai
401	Token tidak ada, rusak, kedaluwarsa, issuer atau audience salah.
403	Token valid, tetapi role tidak cukup, atau melanggar aturan kepemilikan (R6).
404	Resource tidak ada atau berada di luar scope cabang/kepemilikan user . Keberadaannya tidak dibocorkan.
409	Konflik state: transisi status tidak valid, aset tidak AVAILABLE, batas R4/R5, kode duplikat.
422	Input tidak valid (format, due_date di luar batas R3).

Konvensi 404 untuk resource di luar scope adalah default latihan ini. Kalau kamu memilih 403, tulis alasannya di ADR.

Matriks akses

Endpoint	BORROWER	STAFF	BRANCH_MANAGER	SUPER_ADMIN
GET /health	publik	publik	publik	publik
GET /me	✓	✓	✓	✓
GET /assets, /assets/:id	cabang	cabang	cabang	✓
POST /assets	–	–	cabang	✓
PATCH /assets/:id	–	–	cabang	✓
DELETE /assets/:id	–	–	cabang	✓
POST /loans	cabang	–	–	–
GET /loans, /loans/:id	milik sendiri	cabang	cabang	✓
PATCH /loans/:id/approve	–	cabang	cabang	✓
PATCH /loans/:id/reject	–	cabang	cabang	✓
PATCH /loans/:id/handover	–	cabang	cabang	✓
PATCH /loans/:id/return	–	cabang	cabang	✓
PATCH /loans/:id/cancel	milik sendiri	–	–	–
POST /admin/users	–	–	–	✓
PATCH /admin/users/:id/status	–	–	–	✓

Keterangan: ✓ diizinkan, *cabang* hanya cabang sendiri (dari token), *milik sendiri* hanya loan miliknya, – berarti 403. Kolom mengikuti role, sedangkan user dengan beberapa role memakai gabungannya (misal Rina: BORROWER + STAFF).

5.1 Umum

Endpoint	Fungsi dan aturan
GET /health	Publik. Menandakan API hidup (dan koneksi DB berjalan). Tujuan latihan: memakai <code>@Public()</code> sebagai pengecualian AuthGuard global.
GET /me	Mengembalikan profil user yang sedang login: id, email, nama, roles dan branch_code dari token. Request pertama user memicu JIT provisioning. Endpoint paling awal untuk menguji rantai token → guard → DB.

5.2 Assets

Endpoint	Fungsi dan aturan
GET /assets	Daftar aset sesuai scope: non-admin hanya cabangnya, SUPER_ADMIN semua. Query: status, category, page, limit, dan branch_code (hanya berpengaruh untuk SUPER_ADMIN).
GET /assets/:id	Detail satu aset. Di luar scope → 404.
POST /assets	Membuat aset. Body: asset_code, name, category, description. Cabang diambil dari token untuk BRANCH_MANAGER. SUPER_ADMIN wajib mengirim branch_code. asset_code duplikat → 409.
PATCH /assets/:id	Mengubah nama, kategori, deskripsi, dan status secara manual (hanya AVAILABLE, MAINTENANCE, RETIRED). Aset yang sedang RESERVED atau BORROWED tidak boleh diubah statusnya → 409.
DELETE /assets/:id	Soft delete. Aset yang masih punya loan aktif → 409.

5.3 Loans

Endpoint	Fungsi dan aturan
POST /loans	Peminjam mengajukan pinjaman. Body: asset_id, due_date, purpose. Berlaku R1-R5. Hasil: loan REQUESTED (201). borrower_id dari token.
GET /loans	Daftar loan sesuai scope: BORROWER hanya miliknya, STAFF/BRANCH_MANAGER cabangnya, SUPER_ADMIN semua. Query: status, borrower_id (untuk staf ke atas), page, limit.
GET /loans/:id	Detail loan. Di luar scope → 404.
PATCH /loans/:id/approve	REQUESTED → APPROVED, aset → RESERVED, approved_by_id terisi. Satu transaksi + lock (R9). Approver bukan peminjam (R6). Aset tidak AVAILABLE → 409.
PATCH /loans/:id/reject	Body: reason. REQUESTED → REJECTED. Aset tidak berubah. Berlaku R6.
PATCH /loans/:id/handover	APPROVED → BORROWED, aset → BORROWED, borrowed_at dan handed_over_by_id terisi.
PATCH /loans/:id/return	Body: return_notes, needs_maintenance (boolean). BORROWED atau OVERDUE → RETURNED. Aset → AVAILABLE, atau MAINTENANCE bila needs_maintenance true.
PATCH /loans/:id/cancel	Hanya pemilik loan. REQUESTED atau APPROVED → CANCELLED. Aset RESERVED kembali AVAILABLE. Body opsional: reason.

5.4 Admin

Endpoint	Fungsi dan aturan
POST /admin/users	Membuat karyawan baru di Keycloak dan di DB lokal . Body: email, full_name, branch_code, roles (array), temporary_password. Memakai service account. Wajib menangani kegagalan parsial.
PATCH /admin/users/:id/status	Body: enabled (boolean). Meng-enable atau men-disable user di Keycloak, mencabut sesi aktifnya, dan menyamakan users.is_active.

Job terjadwal (bukan endpoint)

Job	Fungsi
OverdueJob	Cron harian (mis. 00:05). Ubah semua loan BORROWED dengan due_date sebelum hari ini menjadi OVERDUE. Buat method-nya bisa dipanggil langsung dari test.

Catatan: login, refresh token, dan logout **tidak** ada di backend. Semuanya dipanggil langsung ke endpoint OpenID Connect Keycloak.

Contoh bentuk request dan respons

GET /me → 200

```
{
  "id": "b1c2d3e4-0000-4000-8000-000000000001",
  "keycloak_id": "3f9c2a6e-0000-4000-8000-000000000002",
  "email": "andi@smartloan.test",
  "full_name": "Andi Pratama",
  "roles": ["BORROWER"],
  "branch_code": "JKT"
}
```

POST /loans → 201

```
// request
{ "asset_id": "<uuid aset JKT>", "due_date": "2026-10-05", "purpose": "Presentasi ke klien" }

// response
{
  "id": "<uuid>", "loan_number": "LN-202609-0001", "status": "REQUESTED",
  "asset_id": "<uuid>", "borrower_id": "<uuid>", "branch_id": "<uuid JKT>",
  "due_date": "2026-10-05", "purpose": "Presentasi ke klien"
}
```

6. Requirement Keycloak

- 1 Realm smartloan dengan brute-force protection aktif, password policy (min. 10 karakter, wajib angka), dan access token lifespan 5 menit.
- 2 Client smartloan-web: public client, Authorization Code + PKCE (disimulasikan lewat Postman/Bruno atau skrip kecil).
- 3 Client smartloan-api: confidential client untuk backend, dengan **audience mapper** agar aud di access token memuat smartloan-api.
- 4 Client smartloan-admin-svc: service account (client credentials) dengan hak minimal dari realm-management (view-users, manage-users, tidak lebih).
- 5 Realm role BORROWER, STAFF, BRANCH_MANAGER (composite → STAFF), SUPER_ADMIN.
- 6 Grup /branches/JKT, /branches/SBY, /branches/MLG, dan claim branch_code di token lewat mapper. Tentukan sendiri apakah dari group membership atau user attribute, lalu tulis alasannya di ADR singkat.
- 7 Seed lewat **realm export/import JSON**, bukan klik manual, supaya environment bisa direproduksi.
- 8 7 user uji sesuai tabel di bagian 2, termasuk satu user *disabled*.

Bentuk access token yang diharapkan (payload, disederhanakan)

```
{
  "iss": "http://localhost:8080/realms/smartloan",
  "sub": "3f9c2a6e-0000-4000-8000-000000000002",
  "aud": "smartloan-api",
  "azp": "smartloan-web",
  "exp": 1790000000,
  "preferred_username": "andi",
  "email": "andi@smartloan.test",
  "name": "Andi Pratama",
  "realm_access": { "roles": ["BORROWER"] },
  "branch_code": "JKT"
}
```

Perhatikan perbedaan aud (siapa yang dituju token ini: API) dan azp (siapa yang meminta token: aplikasi web).

7. Requirement Backend (NestJS + TypeORM)

Struktur modul yang disarankan

Modul	Isi
auth/	Guard JWT via JWKS, RolesGuard, decorator @Public(), @Roles(), @CurrentUser().
keycloak-admin/	Client Admin REST API + cache token service account.
users/	Entity, layanan JIT provisioning, controller /me.
branches/	Entity dan seed.
assets/	Entity, service, controller.
loans/	Entity, service (state machine), controller, OverdueJob.
admin-users/	Controller /admin/users.
database/	Migration dan seed.

Autentikasi

- Validasi JWT via JWKS (cache key, tangani key rotation). Verifikasi iss, aud, exp, dan signature.
- AuthGuard global dengan @Public() untuk pengecualian.
- @CurrentUser() mengembalikan objek terparse (sub, email, roles, branchCode).

Otorisasi

- Role check di guard. **Branch scoping di service/query layer**, dengan nilai cabang dari token.
- Semua aturan R1-R9 ditegakkan di service, bukan di controller.

Data dan transaksi

- Migration, bukan synchronize: true. Validasi input dengan DTO (class-validator).
- JIT provisioning aman terhadap request paralel (unique keycloak_id + upsert/handle konflik).
- Approve memakai transaksi dan pessimistic_write lock pada baris aset.

Integrasi Admin API

- Token service account di-cache dan di-refresh, jangan minta token baru tiap request.
- Tangani kegagalan parsial: user sudah terbentuk di Keycloak tetapi insert lokal gagal.

8. Batasan Keras

- Backend **tidak boleh** menyimpan atau memproses password.
- Tidak ada pengecekan role dengan string hardcoded tersebar di controller. Semua lewat decorator.
- Cabang untuk scoping **selalu dari token**, tidak pernah dari body atau query.
- Tidak ada secret client di repo (gunakan env + `.env.example`).
- Tidak ada panggilan ke Keycloak userinfo di setiap request. Validasi lokal via JWKS.
- Service account tidak boleh diberi role `realm-admin`.
- Tidak memakai `synchronize: true`.
- Satu fase, satu commit/PR.

9. Fase dan Kriteria PASS/FAIL

Fase 1: Fondasi

Lingkup	docker-compose (Keycloak + Postgres), realm import, migration, seed cabang dan aset.
PASS bila	Realm ter-import otomatis dari nol, token 7 user bisa didapat, 4 tabel terbentuk dan 3 cabang ter-seed.

Fase 2: Autentikasi

Lingkup	GET <code>/health</code> , GET <code>/me</code> , guard JWT, JIT provisioning.
PASS bila	<code>/health</code> → 200 tanpa token. <code>/me</code> valid → 200. Token expired/tanpa token/aud salah/realm lain → 401. Request pertama membuat baris users, 10 request paralel pertama tetap menghasilkan satu baris.

Fase 3: RBAC + Assets

Lingkup	Roles guard dan CRUD aset.
PASS bila	BORROWER ke POST <code>/assets</code> → 403. BRANCH_MANAGER JKT → 201 di cabang JKT. Membuka aset SBY → 404. SUPER_ADMIN melihat semua cabang. Composite role bekerja tanpa assign ganda.

Fase 4: Loans (alur normal)

Lingkup	POST <code>/loans</code> , list/detail, approve, reject, handover, return, cancel.
PASS bila	Alur REQUESTED → APPROVED → BORROWED → RETURNED berjalan dan status aset ikut berubah. Tono tidak bisa mengakses loan JKT (404). Dewi tidak bisa meminjam aset JKT (404). Transisi ilegal → 409.

Fase 5: Aturan lanjutan

Lingkup	R4, R5, R6, R9, OverdueJob, partial unique index.
PASS bila	Batas 2 loan aktif dan blokir OVERDUE ditegakkan. Rina tidak bisa approve loan miliknya (403). Job mengubah BORROWED yang lewat <code>due_date</code> menjadi OVERDUE. Dua approve paralel untuk satu aset: satu 200, satu 409.

Fase 6: Admin API

Lingkup	POST <code>/admin/users</code> dan PATCH <code>/admin/users/:id/status</code> .
PASS bila	User baru bisa login (dengan ganti password) dan GET <code>/me</code> tidak menduplikasi profil. Kegagalan insert lokal ditangani. User yang di-disable tidak bisa login dan refresh. Token service account ter-cache.

Fase 7: Sesi**Lingkup** Refresh token flow dan logout.**PASS bila** Refresh menghasilkan token baru. Setelah logout, refresh token tidak bisa dipakai. Access token yang sudah lewat 5 menit → 401.**Fase 8: Test****Lingkup** E2E dengan Keycloak asli (bukan mock JWT).**PASS bila** Semua skenario PASS di fase 1–7 tercakup minimal satu e2e test.

10. Pertanyaan Validasi Pemahaman

Jawab tertulis di README, bukan sekadar lolos test.

- 1 Role user dicabut di Keycloak, tetapi access token lamanya masih valid 4 menit. Apa dampaknya, dan opsi mitigasinya apa saja beserta trade-off-nya?
- 2 Apa beda azp dan aud, dan kenapa keduanya penting untuk backend?
- 3 Kenapa sub lebih tepat sebagai kunci relasi ketimbang email atau preferred_username?
- 4 User dihapus di Keycloak, bagaimana nasib baris users dan loans-nya di DB lokal?
- 5 Apa risikonya kalau branch_code diambil dari body request, bukan dari token?
- 6 Kenapa role tidak disimpan di tabel users, dan apa yang rusak kalau kamu menyimpannya di sana?
- 7 Kenapa scoping cabang memakai claim token dan bukan users.branch_id? Kapan keduanya bisa berbeda?

11. Stretch Goals

- Login via Google (identity brokering) dengan first-login flow.
- Keycloak Authorization Services (resource/policy-based) sebagai pembanding terhadap RBAC manual.
- Event listener atau webhook untuk sinkronisasi saat user dihapus atau di-disable.
- Rate limiting per sub dan tabel audit log (siapa memproses loan apa).

12. Deliverables

Artefak	Isi
docker-compose.yml	Keycloak + Postgres, siap jalan dengan satu perintah
Realm export JSON	Realm, client, role, group, dan 7 user uji
Migration + seed	4 tabel, 3 cabang, aset contoh
README.md	Cara menjalankan + jawaban bagian 10
ADR singkat	Keputusan sumber branch_code (group vs attribute) dan pilihan 403 vs 404
Koleksi Postman/Bruno	Skenario tiap fase, termasuk kasus 401/403/404/409